

Session 5

Question 1

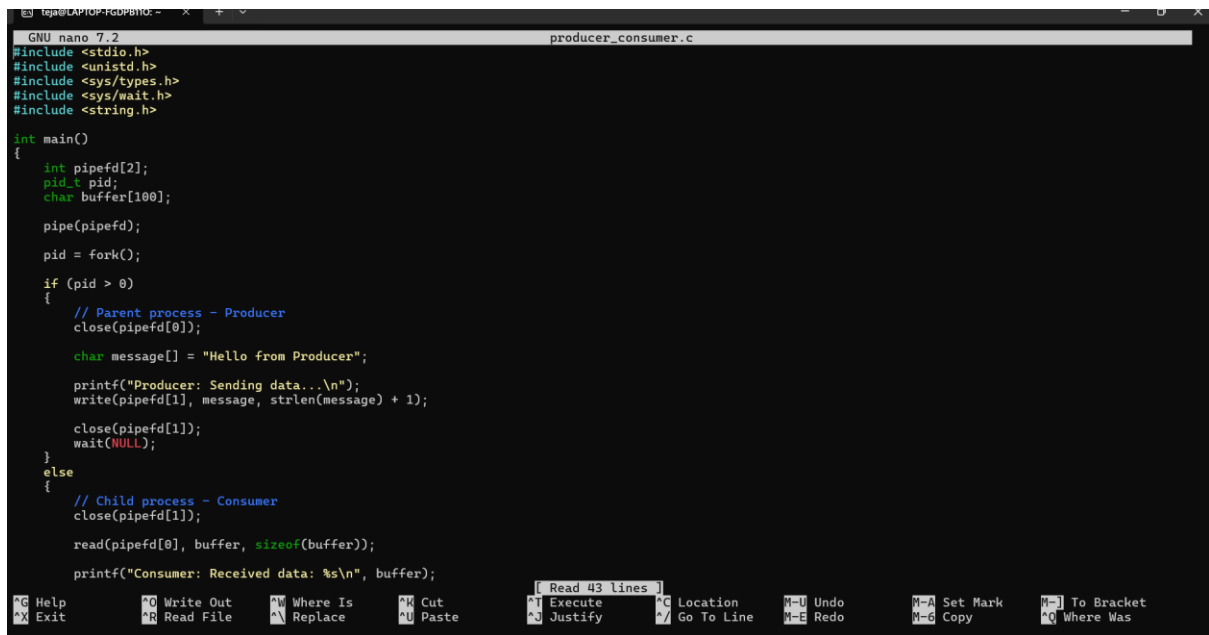
Write a C program to implement the **Producer-Consumer** problem using an anonymous pipe.

Question 2

Write a C program to implement the equivalent of the Linux command:

`ls -l | grep ".c"`

using `fork()`, `pipe()`, `dup2()`, and `execlp()`



```
GNU nano 7.2 producer_consumer.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int pipefd[2];
    pid_t pid;
    char buffer[100];

    pipe(pipefd);

    pid = fork();

    if (pid > 0)
    {
        // Parent process - Producer
        close(pipefd[0]);

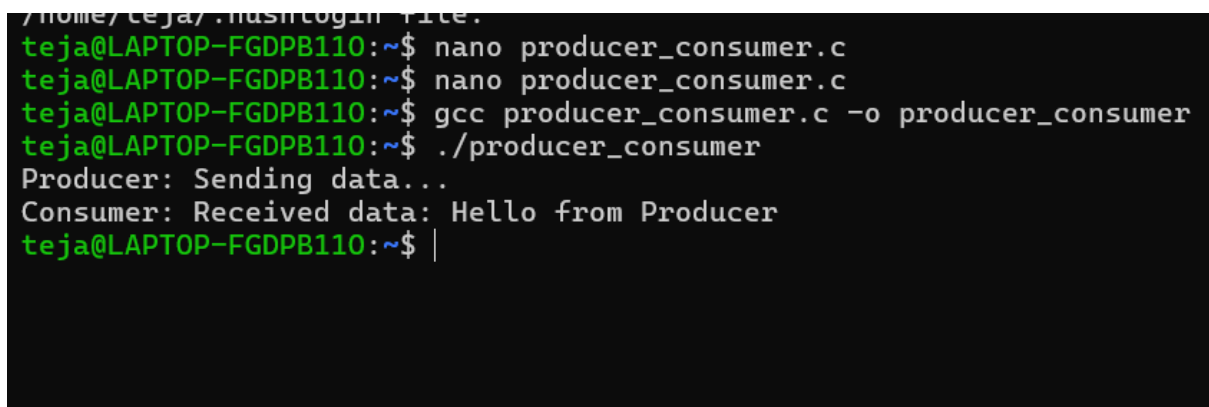
        char message[] = "Hello from Producer";

        printf("Producer: Sending data...\n");
        write(pipefd[1], message, strlen(message) + 1);

        close(pipefd[1]);
        wait(NULL);
    }
    else
    {
        // Child process - Consumer
        close(pipefd[1]);

        read(pipefd[0], buffer, sizeof(buffer));

        printf("Consumer: Received data: %s\n", buffer);
    }
}
```



```
/home/teja/.nushlogin: file.
teja@LAPTOP-FGDPB110:~$ nano producer_consumer.c
teja@LAPTOP-FGDPB110:~$ nano producer_consumer.c
teja@LAPTOP-FGDPB110:~$ gcc producer_consumer.c -o producer_consumer
teja@LAPTOP-FGDPB110:~$ ./producer_consumer
Producer: Sending data...
Consumer: Received data: Hello from Producer
teja@LAPTOP-FGDPB110:~$ |
```

using System Calls

Session 5

```
GNU nano 7.2 shell_equivalent.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid;

    pipe(pipefd);

    pid = fork();

    if (pid == 0)
    {
        // Child process executes ls -l
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp");
        exit(1);
    }
    else
    {
        // Parent process executes grep ".c"
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);
    }
}
```

Memory usage: 6%
Swap usage: 0%

IPv4 address for eth0: 172.18.49.48

This message is shown once a day. To disable it please create the /home/teja/.hushlogin file.

```
teja@LAPTOP-FGDPB110:~$ nano producer_consumer.c
teja@LAPTOP-FGDPB110:~$ nano producer_consumer.c
teja@LAPTOP-FGDPB110:~$ gcc producer_consumer.c -o producer_consumer
teja@LAPTOP-FGDPB110:~$ ./producer_consumer
Producer: Sending data...
Consumer: Received data: Hello from Producer
teja@LAPTOP-FGDPB110:~$ nano shell_equivalent.c
teja@LAPTOP-FGDPB110:~$ nano shell_equivalent.c
teja@LAPTOP-FGDPB110:~$ gcc shell_equivalent.c -o shell_equivalent
teja@LAPTOP-FGDPB110:~$ ./shell_equivalent
-rwxr-xr-x  1 teja teja 16224 Aug 12 05:54 copy
-rw-r--r--  1 teja teja   454 Aug 12 05:52 copy.c
-rw-r--r--  1 teja teja    0 Aug 13 11:37 echo
-rw-r--r--  1 teja teja   683 Aug 12 06:38 history.c
-rw-r--r--  1 teja teja  1366 Aug 12 06:44 memory.c
-rwxr-xr-x  1 teja teja 15960 Aug  5 09:15 process
-rw-r--r--  1 teja teja   80 Aug  5 09:14 process.c
-rw-----  1 teja teja   27 Aug  3 05:49 process.c.save
-rw-----  1 teja teja   11 Aug  3 06:00 process.c.save.1
-rwxr-xr-x  1 teja teja 16360 Aug 26 06:39 producer_consumer
-rw-r--r--  1 teja teja   740 Aug 26 06:39 producer_consumer.c
-rw-r--r--  1 teja teja   703 Aug 26 06:44 shell_equivalent.c
-rw-r--r--  1 teja teja   897 Aug 13 11:25 task1.c
-rw-r--r--  1 teja teja   647 Aug 13 11:31 task2.c
-rw-r--r--  1 teja teja   683 Aug 13 11:36 task3.c
-rw-r--r--  1 teja teja    0 Aug 13 12:53 task4.c
-rw-----  1 teja teja   599 Aug 13 11:44 task4.c.save
-rw-r--r--  1 teja teja   613 Aug 13 12:54 task4.c.save.1
-rw-r--r--  1 teja teja  1179 Aug 13 12:55 task5.c
-rw-r--r--  1 teja teja   685 Aug 22 14:05 week3_task3.c
-rw-r--r--  1 teja teja  1190 Aug 22 13:32 week4_task1.c
-rw-r--r--  1 teja teja   607 Aug 22 13:50 week4_task2.c
-rw-----  1 teja teja   607 Aug 22 13:48 week4_task2.c.save
-rw-----  1 teja teja   607 Aug 22 13:50 week4_task2.c.save.1
teja@LAPTOP-FGDPB110:~$
```

Session 5

```

-rw----- 1 teja teja 607 Aug 22 13:50 week4_task2.c.save.1
teja@LAPTOP-FGDPB110:~$ ./shell_equivalent
-rwxr-xr-x 1 teja teja 16224 Aug 12 05:54 copy
-rw-r--r-- 1 teja teja 454 Aug 12 05:52 copy.c
-rw-r--r-- 1 teja teja 0 Aug 13 11:37 echo
-rw-r--r-- 1 teja teja 683 Aug 12 06:38 history.c
-rw-r--r-- 1 teja teja 1366 Aug 12 06:44 memory.c
-rwxr-xr-x 1 teja teja 15960 Aug 5 09:15 process
-rw-r--r-- 1 teja teja 80 Aug 5 09:14 process.c
-rw----- 1 teja teja 27 Aug 3 05:49 process.c.save
-rw----- 1 teja teja 11 Aug 3 06:00 process.c.save.1
-rwxr-xr-x 1 teja teja 16360 Aug 26 06:39 producer_consumer
-rw-r--r-- 1 teja teja 740 Aug 26 06:39 producer_consumer.c
-rw-r--r-- 1 teja teja 703 Aug 26 06:44 shell_equivalent.c
-rw-r--r-- 1 teja teja 897 Aug 13 11:25 task1.c
-rw-r--r-- 1 teja teja 647 Aug 13 11:31 task2.c
-rw-r--r-- 1 teja teja 683 Aug 13 11:36 task3.c
-rw-r--r-- 1 teja teja 0 Aug 13 12:53 task4.c
-rw----- 1 teja teja 599 Aug 13 11:44 task4.c.save
-rw-r--r-- 1 teja teja 613 Aug 13 12:54 task4.c.save.1
-rw-r--r-- 1 teja teja 1179 Aug 13 12:55 task5.c
-rw-r--r-- 1 teja teja 685 Aug 22 14:05 week3_task3.c
-rw-r--r-- 1 teja teja 1190 Aug 22 13:32 week4_task1.c
-rw-r--r-- 1 teja teja 607 Aug 22 13:50 week4_task2.c
-rw----- 1 teja teja 607 Aug 22 13:48 week4_task2.c.save
-rw----- 1 teja teja 607 Aug 22 13:50 week4_task2.c.save.1
teja@LAPTOP-FGDPB110:~$

```

```

-rw----- 1 teja teja 607 Aug 22 13:50 week4_task2.c.save.1
teja@LAPTOP-FGDPB110:~$ ls
OSLab copy.c memory process.c.save shell_equivalent task2 task4 task4.save week3_task3.c week4_task2.c
OSSP echo memory.c process.c.save.1 shell_equivalent.c task2.c task4.c task4.c task5 week4_task1.c week4_task2.c.save
Week3 history.c process producer_consumer task1 task3.c task4.c task4.c.save task5.c week4_task1.c week4_task2.c.save.1
copy ls process.c producer_consumer.c task1.c
teja@LAPTOP-FGDPB110:~$ ls -l
total 296
drwxr-xr-x 2 teja teja 4096 Aug 8 06:15 OSLab
drwxr-xr-x 11 teja teja 4096 Aug 7 06:04 OSSP
drwxr-xr-x 3 teja teja 4096 Aug 7 05:50 Week3
-rwxr-xr-x 1 teja teja 16224 Aug 12 05:54 copy
-rw-r--r-- 1 teja teja 454 Aug 12 05:52 copy.c
-rw-r--r-- 1 teja teja 0 Aug 13 11:37 echo
-rw-r--r-- 1 teja teja 683 Aug 12 06:38 history.c
-rw-r--r-- 1 teja teja 0 Aug 13 11:28 ls
-rwxr-xr-x 1 teja teja 19248 Aug 12 06:45 memory
-rw-r--r-- 1 teja teja 1366 Aug 12 06:44 memory.c
-rwxr-xr-x 1 teja teja 15960 Aug 5 09:15 process
-rw-r--r-- 1 teja teja 80 Aug 5 09:14 process.c
-rw----- 1 teja teja 27 Aug 3 05:49 process.c.save
-rw----- 1 teja teja 11 Aug 3 06:00 process.c.save.1
-rwxr-xr-x 1 teja teja 16360 Aug 26 06:39 producer_consumer
-rw-r--r-- 1 teja teja 740 Aug 26 06:39 producer_consumer.c
-rwxr-xr-x 1 teja teja 16224 Aug 26 06:44 shell_equivalent
-rw-r--r-- 1 teja teja 703 Aug 26 06:44 shell_equivalent.c
-rwxr-xr-x 1 teja teja 16184 Aug 13 11:26 task1
-rw-r--r-- 1 teja teja 897 Aug 13 11:25 task1.c
-rwxr-xr-x 1 teja teja 16136 Aug 13 11:31 task2
-rw-r--r-- 1 teja teja 647 Aug 13 11:31 task2.c
-rwxr-xr-x 1 teja teja 16104 Aug 13 11:36 task3
-rw-r--r-- 1 teja teja 683 Aug 13 11:36 task3.c
-rw-r--r-- 1 teja teja 599 Aug 13 11:50 task4
-rw-r--r-- 1 teja teja 0 Aug 13 12:53 task4.c
-rw----- 1 teja teja 599 Aug 13 11:44 task4.c.save
-rw-r--r-- 1 teja teja 613 Aug 13 12:54 task4.c.save.1
-rw----- 1 teja teja 599 Aug 13 11:49 task4.save
-rwxr-xr-x 1 teja teja 16480 Aug 13 12:55 task5
-rw-r--r-- 1 teja teja 1179 Aug 13 12:55 task5.c

```

```

-rw-r--r-- 1 teja teja 599 Aug 13 11:50 task4
-rw-r--r-- 1 teja teja 0 Aug 13 12:53 task4.c
-rw----- 1 teja teja 599 Aug 13 11:44 task4.c.save
-rw-r--r-- 1 teja teja 613 Aug 13 12:54 task4.c.save.1
-rw----- 1 teja teja 599 Aug 13 11:49 task4.save
-rwxr-xr-x 1 teja teja 16480 Aug 13 12:55 task5
-rw-r--r-- 1 teja teja 1179 Aug 13 12:55 task5.c
-rwxr-xr-x 1 teja teja 16176 Aug 22 14:06 week3_task3
-rw-r--r-- 1 teja teja 685 Aug 22 14:05 week3_task3.c
-rwxr-xr-x 1 teja teja 16360 Aug 22 13:35 week4_task1
-rw-r--r-- 1 teja teja 1190 Aug 22 13:32 week4_task1.c
-rwxr-xr-x 1 teja teja 16216 Aug 22 13:51 week4_task2
-rw-r--r-- 1 teja teja 607 Aug 22 13:50 week4_task2.c
-rw----- 1 teja teja 607 Aug 22 13:48 week4_task2.c.save
-rw----- 1 teja teja 607 Aug 22 13:50 week4_task2.c.save.1
teja@LAPTOP-FGDPB110:~$ ls -l | grep ".c"
-rwxr-xr-x 1 teja teja 16224 Aug 12 05:54 copy
-rw-r--r-- 1 teja teja 454 Aug 12 05:52 copy.c
-rw-r--r-- 1 teja teja 0 Aug 13 11:37 echo
-rw-r--r-- 1 teja teja 683 Aug 12 06:38 history.c
-rw-r--r-- 1 teja teja 1366 Aug 12 06:44 memory.c
-rwxr-xr-x 1 teja teja 15960 Aug 5 09:15 process
-rw-r--r-- 1 teja teja 80 Aug 5 09:14 process.c
-rw----- 1 teja teja 27 Aug 3 05:49 process.c.save
-rw----- 1 teja teja 11 Aug 3 06:00 process.c.save.1
-rwxr-xr-x 1 teja teja 16360 Aug 26 06:39 producer_consumer
-rw-r--r-- 1 teja teja 740 Aug 26 06:39 producer_consumer.c
-rw-r--r-- 1 teja teja 703 Aug 26 06:44 shell_equivalent.c
-rw-r--r-- 1 teja teja 897 Aug 13 11:25 task1.c
-rw-r--r-- 1 teja teja 647 Aug 13 11:31 task2.c
-rw-r--r-- 1 teja teja 683 Aug 13 11:36 task3.c
-rw-r--r-- 1 teja teja 0 Aug 13 12:53 task4.c
-rw----- 1 teja teja 599 Aug 13 11:44 task4.c.save
-rw-r--r-- 1 teja teja 613 Aug 13 12:54 task4.c.save.1
-rw-r--r-- 1 teja teja 1179 Aug 13 12:55 task5.c
-rw-r--r-- 1 teja teja 685 Aug 22 14:05 week3_task3.c
-rw-r--r-- 1 teja teja 1190 Aug 22 13:32 week4_task1.c
-rw-r--r-- 1 teja teja 607 Aug 22 13:50 week4_task2.c
-rw----- 1 teja teja 607 Aug 22 13:48 week4_task2.c.save
-rw----- 1 teja teja 607 Aug 22 13:50 week4_task2.c.save.1
teja@LAPTOP-FGDPB110:~$ |

```

A **pipe** is an Inter-Process Communication (IPC) mechanism used to exchange data between processes. A pipe has two ends: one for reading and one for writing.

The `fork()` system call creates a new child process. In the Producer-Consumer program, the parent process acts as the producer and writes data into the pipe, while the child process acts as the consumer and reads the data from the pipe.

The `pipe()` system call creates the communication channel. The `write()` system call sends data through the pipe, and the `read()` system call receives the data.

The `dup2()` system call is used to redirect standard input or output to the pipe. The `execlp()` system call is used to execute commands such as `ls` and `grep`.

Session 5

The command `ls -l | grep ".c"` displays detailed information about files whose names contain `.c`. The output of `ls -l` is passed through the pipe to `grep`.

Thus, pipes provide a simple and effective way for processes to communicate and exchange data.